NLP LAB EXPERIMENTS

NAME: A. Abhishek Reddy

Rg no:192424192

COURSE CODE: DSA0311

COURSE NAME: NATURAL LANGUAGE PROCESSING

11 Implement a simple top-down parser for context-free grammars using python.

Aim: To implement a simple top-down parser for a context-free grammar using Python.

Algorithm

1. Start.
2. Define a simple grammar.
3. Read a sentence.
4. Split the sentence into words.
5. Check whether the sentence matches the grammar.
6. Display Accepted or Rejected.
7. Stop.

CODE

```
def parse(sentence):
    words = sentence.split()
    if len(words) == 2 and words[0] == "the" and words[1] == "cat":
        print("Sentence Accepted")
    else:
        print("Sentence Rejected")
sentence = input("Enter a sentence: ")
parse(sentence)
```

OUTPUT

```
Enter a sentence: the cat
Sentence Accepted
```

Result: The program checks whether a sentence matches the simple grammar pattern the cat.

12 Implement an Earley parser for context-free grammars using a simple python program.

Aim: To implement a simple Earley parser for a context-free grammar using Python.

Algorithm

1. Start.
2. Import the NLTK library.
3. Define a context-free grammar.
4. Create an Earley parser.
5. Read and tokenize a sentence.
6. Parse the sentence using the grammar.
7. Display Accepted or Rejected.
8. Stop.

CODE

```
import nltk
grammar = nltk.CFG.fromstring("""
S -> NP VP | NP
NP -> Det N
Det -> 'the' | 'The'
N -> 'cat' | 'dog'
VP -> 'runs' | 'plays'
""")
parser = nltk.EarleyChartParser(grammar)
sentence = input("Enter a sentence: ").split()
try:
    if any(parser.parse(sentence)):
        print("Accepted")
    else:
        print("Rejected")
except ValueError as e:
    print("Error:", e)
    print("Rejected (Word not in grammar)")
```

OUTPUT

```
Enter a sentence: the dog
Accepted
```

Result: The program uses an Earley parser to check whether a sentence follows the given context-free grammar.

13 Generate a parse tree for a given sentence using a context-free grammar using python program.

Aim: To generate a parse tree for a given sentence using a context-free grammar in Python.

Algorithm

1. Start.
2. Define a context-free grammar.
3. Create a chart parser.
4. Read a sentence and tokenize it.
5. Generate the parse tree.
6. Display the parse tree.
7. Stop.

CODE

```
import nltk
grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> 'the' 'cat'
VP -> 'runs'
""")
parser = nltk.ChartParser(grammar)
sentence = "the cat runs".split()
for tree in parser.parse(sentence):
    print(tree)
    tree.pretty_print()
```

OUTPUT:

```
(S (NP the cat) (VP runs))
          S
       ___|___
      NP      VP
   ___|___    |
  the     cat runs
```

Result: The program generates and displays a parse tree for the given sentence using a context-free grammar.

14 Create a program in python to check for agreement in sentences based on a context-free grammar's rules.

Aim: To check sentence agreement using context-free grammar rules in Python.

Algorithm

1. Start.
2. Read the subject and verb.
3. Check the subject–verb agreement rules.
4. If the subject and verb agree, display Correct.
5. Otherwise, display Incorrect.
6. Stop.

CODE

```
subject = input("Enter subject (He/They): ")
verb = input("Enter verb (run/runs): ")
if (subject == "He" and verb == "runs") or (subject == "They" and verb == "run"):
    print("Correct sentence")
else:
    print("Incorrect sentence")
```

OUTPUT

```
Enter subject (He/They): He
Enter verb (run/runs): runs
Correct sentence
```

Result: The program checks subject–verb agreement in a sentence.

15 Implement probabilistic context-free grammar parsing for a sentence using python.

Aim: To implement probabilistic context-free grammar parsing for a sentence using Python.

Algorithm

1. Start.
2. Import the NLTK library.
3. Define a probabilistic context-free grammar with probabilities.
4. Create a Viterbi parser.
5. Read and tokenize a sentence.
6. Parse the sentence and calculate its probability.
7. Display the parse tree and probability.
8. Stop.

CODE

```
import nltk
grammar = nltk.PCFG.fromstring("""
S -> NP VP [1.0]
NP -> 'the' 'cat' [1.0]
VP -> 'runs' [1.0]
""")
parser = nltk.ViterbiParser(grammar)
sentence = "the cat runs".split()
for tree in parser.parse(sentence):
    print(tree)
```

OUTPUT

```
(S (NP the cat) (VP runs)) (p=1)
```

Result: The program identifies named entities such as persons, locations, and organizations in the given text.

16.Implement a Python program using the SpaCy library to perform Named Entity Recognition (NER) on a given text.

Aim: To identify named entities in a given text using the spaCy library in Python.

Algorithm

1. Start.
2. Import the spaCy library.
3. Load the English language model.
4. Read the input text.
5. Identify named entities in the text.
6. Display the entities and their labels.
7. Stop.

CODE

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = "Ravi lives in Chennai and works at Google."

doc = nlp(text)

for ent in doc.ents:

   print(ent.text, "-", ent.label_)
```

OUTPUT

```
Chennai - ORG
Google - ORG
```

Result: The program identifies named entities such as persons, locations, and organizations in the given text.

17 Write program demonstrates how to access WordNet, a lexical database, to retrieve synsets and explore word meanings in python.

Aim: To retrieve synsets and explore word meanings using WordNet in Python.

Algorithm

1. Start.

2. Import WordNet from NLTK.
3. Read a word from the user.
4. Retrieve its synsets from WordNet.
5. Display the meanings of the word.
6. Stop.

CODE

```
from nltk.corpus import wordnet
word = input("Enter a word: ")
for syn in wordnet.synsets(word):
   print("Meaning:", syn.definition())
```

OUTPUT

```
Enter a word: happy
Meaning: enjoying or showing or marked by joy or pleasure
Meaning: marked by good fortune
Meaning: eagerly disposed to act or to be of service
Meaning: well expressed and to the point
```

Result: The program retrieves different meanings of a word from the WordNet lexical database.

18 Implement a simple FOPC parser for basic logical expressions using python program.

Aim: To implement a simple First-Order Predicate Calculus (FOPC) parser for basic logical expressions using Python.

Algorithm

1. Start.
2. Import the regular expression library.
3. Read a logical expression.
4. Check whether it follows the required predicate format.
5. Display Valid or Invalid.
6. Stop.

CODE

```
import re
expression = input("Enter a logical expression: ")
```

```
if re.match(r"^[A-Za-z]+\([A-Za-z]+\)$", expression):
    print("Valid logical expression")
else:
    print("Invalid logical expression")
```

OUTPUT

```
Enter a logical expression: Human(Ravi)
Valid logical expression
```

Result: The program checks whether a basic predicate expression follows the required format, such as Predicate(Constant).

19 Create a program for word sense disambiguation using the Lesk algorithm using python.

Aim: To identify the correct meaning of a word using the Lesk algorithm in Python.

Algorithm

1. Start.
2. Import the Lesk algorithm from NLTK.
3. Read a sentence containing an ambiguous word.
4. Identify the intended word.
5. Apply the Lesk algorithm using the sentence context.
6. Display the selected meaning.
7. Stop.

CODE

```
import nltk
from nltk.wsd import lesk
nltk.download('wordnet')
sentence = "I went to the bank to deposit money".split()
meaning = lesk(sentence, 'bank')
print("Word:", "bank")
print("Meaning:", meaning.definition() if meaning else "Meaning not found")
```

OUTPUT

```
Word: bank
Meaning: a container (usually with a slot in the top) for keeping money at home
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

Result: The program uses the Lesk algorithm to identify the appropriate meaning of a word based on its context.

20 Implement a basic information retrieval system using TF-IDF (Term Frequency-Inverse Document Frequency) for document ranking using python.

Aim: To implement a basic information retrieval system using TF-IDF to rank documents in Python.

Algorithm

1. Start.
2. Import the TF-IDF vectorizer and cosine similarity functions.
3. Define a collection of documents.
4. Read the search query.
5. Calculate TF-IDF vectors for documents and query.
6. Calculate similarity scores.
7. Rank documents by similarity.
8. Display the ranked documents.
9. Stop.

CODE

```
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity

docs = ["Python is easy", "Python is powerful", "Java is popular"]

query = input("Enter search query: ")

vectorizer = TfidfVectorizer()

data = vectorizer.fit_transform(docs + [query])

scores = cosine_similarity(data[-1], data[:-1])[0]

for i in scores.argsort()[::-1]:
```

```
print(docs[i], "-", round(scores[i], 2))
```

OUTPUT

```
Enter search query: Python
Python is powerful - 0.47
Python is easy - 0.47
Java is popular - 0.0
```

Result: The program ranks documents according to their similarity to the search query using TF-IDF and cosine similarity.